A Mathematically Consistent, Nengo-Compatible Reference Implementation of HyperLIF and Adaptive HyperLIF Neurons: Continuous Gate-State Dynamics, Exact Validation, and a Bridge to Gate-State Residual Decoding

Adaptive HyperLIF / Nengo Research Project

***Abstract*—The leaky integrate-and-fire (LIF) neuron resets its membrane potential discontinuously at threshold, which is convenient for simulation but makes the state evolution non-smooth and obscures the sub-threshold "readiness" state of the neuron. The companion theory manuscript [?] replaces the hard reset with a continuous, gate-controlled discharge, yielding the *HyperLIF* neuron and its input-stabilized variant, *Adaptive HyperLIF*. This paper contributes a clean, self-contained, and rigorously validated reference implementation of those neurons that is compatible with the Neural Engineering Framework toolkit Nengo [?]. We restate the implemented equations exactly as coded, expose two numerical stepping schemes—an explicit-Euler scheme matching the Julia reference and an exponential-Euler scheme used by the Nengo port—and analyze their accuracy and stability against a fine Runge–Kutta reference. A frozen-open-gate equilibrium analysis gives the branch estimate $J_{\mathrm{DB}} = (v_{\mathrm{thresh}} - \gamma)(1+\tau G_b) = 2.4$ for the reference parameters and the branch voltage $v^* = J/(1+\tau G_b)$; the voltage prediction matches the simulated high-drive latch (e.g. $v^* = 1.3333$ predicted and observed at $J = 4$). The adaptive conductance law bounds the positive-drive, $r = 1$ asymptotic branch below the gate knee when $\beta > 1/(\tau(v_{\mathrm{thresh}} - \gamma)) = 62.5$ (here $\beta = 80$). This changes the high-drive regime but is not a global-stability or firing theorem. We further show that the standalone exponential stepping and the Nengo custom-neuron stepping agree to floating-point exactness (voltage and gate RMSE $= 0$, zero event-timing mismatch) across two stimulus protocols and both models, because both call an identical numerical kernel. Finally, we interpret the continuous reset gate as a decodable readiness-to-fire signal and outline—explicitly as future work—how it can serve as a residual feature for Nengo-based adaptive quaternion attitude control. All figures, tables, and claims are reproducible from the included code and data.**

***Index Terms*—Spiking neural networks, leaky integrate-and-fire model, continuous reset, depolarization block, Neural Engineering Framework, Nengo, numerical integration, neuromorphic computing.**

I. Introduction

THE discontinuous threshold-and-reset rule of the LIF neuron [?] makes spike generation a hybrid (non-smooth) dynamical process. This is the root cause of two practical difficulties. First, the reset map is not differentiable, so gradient-based training of spiking networks must either approximate the missing derivative with surrogate gradients [?] or handle spike events explicitly with specialized adjoint machinery [?]. Second, the instantaneous jump discards information: the moment the membrane reaches threshold, its trajectory is overwritten, leaving only the emitted event as an observable. The continuous-reset *HyperLIF* family introduced in the companion theory manuscript [?] removes the discontinuity by modeling the reset as a fast but smooth, gate-controlled discharge, producing a continuous-state ordinary differential equation (ODE) system with an explicit, interpretable internal gate state $r(t)$. The binary event observable remains non-differentiable; event-dependent losses require a surrogate or an explicit event-time derivative. Adaptive HyperLIF also contains a non-smooth $|J|$ term at $J = 0$ unless it is regularized.

Theory and benchmark performance for the HyperLIF family are reported in [?]. The present paper addresses a complementary and equally important question for downstream use: *can these continuous-reset neurons be implemented as a clean, equation-faithful, and numerically validated component of a standard neural-engineering workflow?* We answer this affirmatively by building a small reference package that (i) implements LIF, HyperLIF, and Adaptive HyperLIF from a single mathematical specification, (ii) ports HyperLIF and Adaptive HyperLIF to Nengo [?] as custom `NeuronType` classes with probeable continuous state, and (iii) validates the port against the standalone implementation to floating-point exactness. The Neural Engineering Framework (NEF) [?] that underlies Nengo provides a principled route from encoded stimuli to decoded estimates, and has been used for closed-loop adaptive robot control [?]; a validated HyperLIF `NeuronType` is the prerequisite for using the continuous gate state inside that workflow.

Contributions

This paper makes the following contributions.

1) A single-source, equation-faithful reference implementation of LIF, HyperLIF, and Adaptive HyperLIF, with every manuscript equation traceable to specific lines of code (**????????**).

Companion implementation and validation study to the Adaptive HyperLIF theory manuscript [?]. This manuscript is submitted for review; author identities and affiliations have been anonymized for double-blind peer review.

2) A closed-form characterization of the frozen open-gate equilibrium: the branch estimate $J_{\mathrm{DB}} = (v_{\mathrm{thresh}} - \gamma)(1 + \tau G_b)$ and branch voltage $v^* = J/(1 + \tau G_b)$, checked against high-drive simulation, and the condition $\beta > 1/(\tau(v_{\mathrm{thresh}} - \gamma))$ under which the adaptive positive-drive, $r = 1$ asymptote lies below the gate knee (**??????**).
3) An analysis of the two discrete stepping schemes (explicit-Euler vs. exponential-Euler) against a Runge–Kutta reference, quantifying convergence and contractivity of its frozen scalar exponential substep (**????**).
4) A demonstration that the Nengo port reproduces the standalone exponential stepping *exactly* (voltage/gate RMSE $= 0$, zero event mismatch) across protocols and models, with an explanation grounded in a shared numerical kernel (**????**).
5) An interpretation of the reset gate as a decodable readiness-to-fire signal, and an explicitly-labeled future-work bridge to gate-state residual decoding for quaternion attitude control (**????**).

II. Motivation and Background

A. From Hard Reset to Continuous Reset

A standard LIF neuron integrates its input and, upon crossing v_{thresh}, is reset to v_{reset} instantaneously. The companion theory [**?**] reframes this jump as the standard-real projection of an infinitesimal-time discharge, producing a stiff two-time-scale ODE in which a fast gate variable $r(t)$ mediates a continuous reset conductance. Enforcing continuity exposes dynamical regimes that the idealized hybrid abstraction hides—most notably a depolarization block (DB), a silent depolarized mode under sustained strong drive that is well documented in biophysical models [**?**], [**?**] and is the continuous-system resolution of a Zeno-like accumulation of reset events [**?**]. The Adaptive HyperLIF changes the high-drive branch over the intended operating range by scaling the reset conductance with input magnitude. We take these models as given and focus on a faithful, validated realization.

B. Why Nengo Compatibility Matters

Nengo [**?**] implements the NEF [**?**]: populations of neurons encode signals through tuning curves and decode them through learned linear weights. Implementing HyperLIF as a Nengo `NeuronType` places the continuous-reset neuron inside this standard ecosystem—probeable state, calibrated gains/biases, and ensemble decoding—so that the gate state can be used as a decodable feature alongside spikes and voltage. Because NEF-based spiking controllers have been demonstrated for adaptive robotic control [**?**], a validated port is the natural substrate for the attitude-control direction discussed in **??**.

C. Notation

?? fixes the notation, which is identical to the symbols used in the code and in [**?**]. Unless stated otherwise we use the normalized convention $v_{\mathrm{rest}} = v_{\mathrm{reset}} = 0$, $v_{\mathrm{thresh}} = 1$, so J and the conductance G are dimensionless relative to the leak term.

TABLE I
Notation used throughout the paper.

Symbol	Meaning
$v(t)$	Membrane voltage (normalized)
$r(t) \in [0, 1]$	Continuous reset gate (fast activation-like proxy)
$J\ (= I_{\mathrm{in}})$	Net input current / drive (normalized)
τ, τ_r	Membrane and gate time constants ($\tau_r \ll \tau$)
$v_{\mathrm{rest}}, v_{\mathrm{reset}}, v_{\mathrm{thresh}}$	Rest, reset, threshold voltages
$\sigma(\cdot)$	Logistic gate detector, steepness α
$\gamma, \theta_{\mathrm{eff}}$	Hysteresis and effective threshold $\theta_{\mathrm{eff}} = v_{\mathrm{thresh}} - \gamma r$
G_b	Fixed reset gain (baseline HyperLIF)
G_{base}, β	Baseline gain and adaptive scaling (Adaptive)
$G(r, J)$	Reset conductance scale; $G_b r^2$ or $(G_{\mathrm{base}} + \beta\lvert J\rvert)r^2$
G_{dyn}	Adaptive conductance scale $G_{\mathrm{base}} + \beta\lvert J\rvert$
λ	Membrane relaxation rate $1/\tau + G$
J_{DB}	Frozen-open-gate DB branch estimate
Δt	Simulation time step

III. Mathematical Formulation

All three neuron models share the membrane/leak structure; they differ in how the reset is realized. We state each model exactly as implemented. A compact qualitative comparison is given in **??**, and the default parameters (reflected directly from the parameter dataclasses) in **??**.

A. LIF Baseline

The baseline LIF membrane obeys

$$\frac{dv}{dt} = \frac{-(v - v_{\mathrm{rest}}) + J}{\tau}. \tag{1}$$

For a sub-threshold step the implementation advances the membrane with the exact linear update

$$v_{k+1} = (v_{\mathrm{rest}} + J) + \big(v_k - v_{\mathrm{rest}} - J\big)\, e^{-\Delta t/\tau}, \tag{2}$$

which is the closed-form solution of **??** for constant J. If the membrane crosses threshold from below, $v_k < v_{\mathrm{thresh}} \le v_{k+1}$, the neuron emits an event of magnitude $A/\Delta t$ (Nengo's spike convention), is reset to v_{reset}, and enters a refractory interval τ_{ref} during which the voltage is held at v_{reset}. The voltage is clamped from below at $v_{\min}$. This is a deliberately minimal hard-reset reference for figures and decoding comparisons, not a replacement for Nengo's production LIF.

B. HyperLIF Model

HyperLIF augments the membrane with a continuous reset gate $r(t) \in [0, 1]$. The gate detector is a steep logistic with effective threshold $\theta_{\mathrm{eff}} = v_{\mathrm{thresh}} - \gamma r$ (a hysteresis that lowers the threshold as the gate opens):

$$\theta_{\mathrm{eff}} = v_{\mathrm{thresh}} - \gamma r, \tag{3}$$

$$r_\infty = \sigma\big(\alpha(v - \theta_{\mathrm{eff}})\big), \quad \sigma(x) = \frac{1}{1 + e^{-x}}. \tag{4}$$

The two-state continuous-time dynamics are

$$\frac{dr}{dt} = \frac{r_\infty - r}{\tau_r}, \tag{5}$$

$$\frac{dv}{dt} = \frac{-(v - v_{\mathrm{rest}}) + J}{\tau} - G(r)\,(v - v_{\mathrm{reset}}), \;\; G(r) = G_b\, r^2. \tag{6}$$

TABLE II
QUALITATIVE COMPARISON OF THE THREE NEURON MODELS IN THE REFERENCE IMPLEMENTATION.

Property	LIF	HyperLIF	Adaptive HyperLIF
State variables	$\{v\}$	$\{v, r\}$	$\{v, r\}$
Reset mechanism	hard jump to v_{reset}	gate conductance $G_b r^2$	input-modulated $(G_{\text{base}} + \beta\|J\|)r^2$
Threshold event	discontinuous	smooth crossing	smooth crossing
Continuous & differentiable	no (hybrid)	yes	yes
Refractory handling	explicit τ_{ref}	gate hysteresis γ	gate hysteresis γ
High-drive behaviour	increasing firing	high-voltage latch	bounded subthreshold dynamics
Nengo `NeuronType`	(built-in)	`HyperLIFNengo`	`AdaptiveHyperLIFNengo`

The reset conductance $G(r) = G_b r^2$ is quadratic in the gate so that $\partial G/\partial r = 0$ at gate onset, preserving a sharp, LIF-like upstroke until the gate commits [**?**]. Crucially, *there is no hard reset*: **??** pulls the membrane toward v_{reset} continuously through $G(r)$. Events are emitted for readout only, on an upward crossing of an event level v_{event} (by default v_{thresh}), and do not alter the state.

a) Open-gate equilibrium and the depolarization block: Setting $dv/dt = 0$ in **??** for a frozen gate value r and constant J gives the membrane equilibrium

$$v^*(r) = \frac{v_{\text{rest}} + J + \tau\, G\, v_{\text{reset}}}{1 + \tau\, G}, \qquad G = G_b r^2, \tag{7}$$

i.e. the drive divided by the total (leak plus reset) conductance. Under strong sustained drive the gate saturates open ($r \to 1$). In the normalized convention $v^*(1) = J/(1 + \tau G_b)$. Equating this *frozen open-gate branch* with the gate knee $\theta_{\text{eff}}(1) = v_{\text{thresh}} - \gamma$ gives the useful branch estimate $v^*(1) \geq v_{\text{thresh}} - \gamma$. Solving for the corresponding current,

$$\boxed{J_{\text{DB}} = (v_{\text{thresh}} - \gamma)\,(1 + \tau G_b) - v_{\text{rest}} - \tau G_b\, v_{\text{reset}}.} \tag{8}$$

For the reference parameters ($v_{\text{thresh}} = 1, \gamma = 0.2, \tau = 0.02, G_b = 100$), **??** gives $J_{\text{DB}} = 0.8 \times 3 = 2.4$, and the latched voltage at, e.g., $J = 4$ is $v^*(1) = 4/3 = 1.3333$. **??** confirms both numbers against simulation.

C. Adaptive HyperLIF Model

Adaptive HyperLIF keeps **??????** unchanged and replaces the fixed reset gain by an input-dependent conductance scale,

$$G_{\text{dyn}} = G_{\text{base}} + \beta\,|J|, \quad G(r, J) = G_{\text{dyn}}\, r^2 = (G_{\text{base}} + \beta\,|J|)\, r^2, \tag{9}$$

so that the membrane equation becomes

$$\frac{dv}{dt} = \frac{-(v - v_{\text{rest}}) + J}{\tau} - (G_{\text{base}} + \beta\,|J|)\, r^2\,(v - v_{\text{reset}}). \tag{10}$$

The feedforward term $\beta|J|$ raises the available discharge capacity in proportion to the drive. Substituting $G = G_{\text{base}} + \beta|J|$ with $r = 1$ into **??** and taking $|J| \to \infty$ gives the asymptotic open-gate voltage

$$v^*_\infty = \frac{1 + \tau\beta\, v_{\text{reset}}}{\tau\beta} \xrightarrow{v_{\text{reset}}=0} \frac{1}{\tau\beta}. \tag{11}$$

If this branch limit stays below the gate knee, a fully open gate is not self-consistent on the same high-voltage latch branch as the fixed-gain model. The corresponding normalized branch condition is

$$\boxed{\beta > \frac{1}{\tau\,(v_{\text{thresh}} - \gamma)}.} \tag{12}$$

For the reference parameters the critical value is $1/(0.02 \times 0.8) = 62.5$; the configured $\beta = 80$ satisfies **??**, and indeed $v^*_\infty = 1/(0.02 \times 80) = 0.625 < 0.8$. **??** bounds the positive-drive, $r = 1$ asymptotic equilibrium branch. It does not by itself prove global stability, existence of a limit cycle, or successful discharge for arbitrary time-varying or recurrent input; those properties require analysis of the full coupled system.

IV. NUMERICAL INTEGRATION

The gate dynamics **??** are stiff ($\tau_r \ll \tau$), so naive explicit integration of the gate is inaccurate at practical step sizes. The implementation therefore advances the gate with the *exact* first-order relaxation obtained by freezing the target r_∞ over one step,

$$r_{k+1} = \text{clip}\Big(r_\infty + (r_k - r_\infty)\, e^{-\Delta t/\tau_r},\ 0,\ 1\Big), \tag{13}$$

and then advances the membrane with one of two schemes that share this gate update. The **exponential ("stable") scheme** solves the membrane equation exactly over one step, treating J and the post-update conductance G as constant. Writing the membrane equation as $\dot v = d - \lambda v$ with

$$\lambda = \tfrac{1}{\tau} + G, \qquad d = \tfrac{v_{\text{rest}}}{\tau} + \tfrac{J}{\tau} + G\, v_{\text{reset}}, \tag{14}$$

its exact update is

$$v_{k+1} = v_k\, e^{-\lambda\Delta t} + \frac{d}{\lambda}\left(1 - e^{-\lambda\Delta t}\right). \tag{15}$$

The **explicit-Euler ("Julia") scheme** instead advances the membrane with a forward-Euler step,

$$v_{k+1} = v_k + \Delta t\left[\frac{-(v_k - v_{\text{rest}}) + J}{\tau} - G\,(v_k - v_{\text{reset}})\right]. \tag{16}$$

For frozen J and G, **??** is the exact contractive scalar substep (the factor $e^{-\lambda\Delta t} \in (0, 1]$ for all $\Delta t > 0$), whereas **??** is stable only for $\lambda\Delta t < 2$. This does not establish A-stability or global stability of the nonlinear coupled operator-split map. Both schemes converge to the same continuous trajectory as $\Delta t \to 0$ under the usual consistency and regularity conditions. **??** summarizes the per-step update; it is exactly the code path used by both the standalone simulator (`method="stable"`) and the Nengo port.

Algorithm 1 HyperLIF / Adaptive HyperLIF discrete update (one step).

Require: state (v, r), input J, step Δt, parameters

$\theta_{\text{eff}} \leftarrow v_{\text{thresh}} - \gamma\, r$ ▷ hysteresis

$r_\infty \leftarrow \sigma(\alpha(v - \theta_{\text{eff}}))$ ▷ detector

$r \leftarrow \text{clip}\big(r_\infty + (r - r_\infty)e^{-\Delta t/\tau_r}, 0, 1\big)$

$G \leftarrow G_b r^2$ **or** $(G_{\text{base}} + \beta|J|)r^2$ ▷ HyperLIF / Adaptive

$\lambda \leftarrow 1/\tau + G; \quad d \leftarrow v_{\text{rest}}/\tau + J/\tau + G\, v_{\text{reset}}$

$v_{\text{prev}} \leftarrow v$

$v \leftarrow v\, e^{-\lambda\Delta t} + (d/\lambda)(1 - e^{-\lambda\Delta t})$ ▷ exp.; Euler: **??**

$v \leftarrow \max(v, v_{\min})$

$\text{spike} \leftarrow \big[\, v_{\text{prev}} < v_{\text{event}} \le v \,\big]$ ▷ readout only

return (v, r), spike

V. Nengo Implementation

Nengo 4.1 realizes a custom neuron through `NeuronType.state`: the builder allocates one signal per declared state variable and calls `step(dt, J, output, **state)` at every simulation tick. The ports declare explicit state dictionaries—{`voltage`, `reset_gate`, `previous_voltage`} for `HyperLIFNengo`, plus `dynamic_gain` for `AdaptiveHyperLIFNengo`—all of which are probeable. The body of `step` performs precisely lines 1–9 of **??**: it computes the effective threshold, the logistic target, the exact gate relaxation **??**, the conductance, and the exponential membrane update **??** through the same `stable_voltage_update` routine that the standalone simulator calls. Consequently the Nengo neuron is not a re-derivation but a re-use of one numerical kernel, which is why the agreement reported in **??** is exact rather than merely close.

For ensemble use, the ports also implement NEF gain/bias calibration: the threshold current is $v_{\text{thresh}} - v_{\text{rest}}$, and `rate_to_current` maps a requested maximum firing rate to a top input current, so that standard `max_rates`/`intercepts` specifications produce the expected tuning curves. This calibration is not exercised by the single-neuron validation in this paper (which injects current directly) but is required for the ensemble decoding envisioned in **??**.

VI. Validation Experiments

All experiments use a single neuron driven by a prescribed current at $\Delta t = 10^{-4}$ s unless noted, with the default parameters of **??**. The standalone simulators use `method="julia"` for the dynamics figures (matching the reference stepping) and `method="stable"` for the Nengo parity check. The protocols are: a single *step* ($J = 1.4$ on $[0.03, 0.20]$ s); a piecewise-constant *stress* current ramping to $J = 4$; a constant-current *sweep* over $J \in [0, 6]$ at spacing 0.1 with the firing rate measured over a settled window $[0.15, 0.6]$ s; a sustained *DB-drive* at $J = 3.3$; a smooth multi-frequency *decoding* current; and a numerical-*integration* study at $J = 1.5$ against a Runge–Kutta reference at $\Delta t_{\text{ref}} = 2 \times 10^{-6}$ s. Each experiment writes a CSV and a log; every figure and table in **??** is generated from those CSVs and is traceable to a named file (see the accompanying `FIGURE_TABLE_MAP.md`).

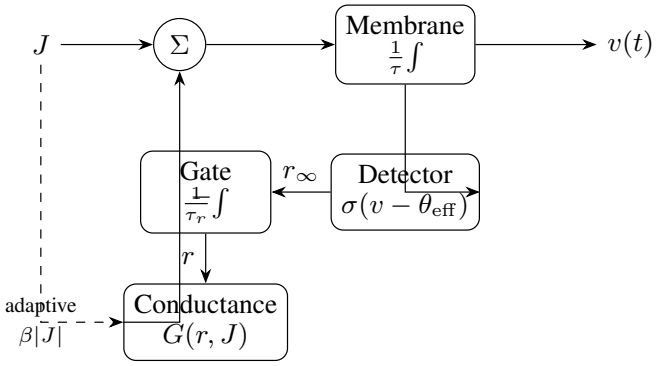


Fig. 1. Signal flow of the (Adaptive) HyperLIF neuron as implemented. The forward path integrates the membrane; the feedback loop (detector → gate → conductance) implements the continuous reset through $r(t)$. The dashed feedforward path is the adaptive term $\beta|J|$ that scales the reset conductance with input magnitude (**??**); removing it recovers the fixed-gain HyperLIF.

VII. Results

A. Continuous vs. Hard Reset

?? contrasts LIF and HyperLIF under the same current step. The LIF membrane produces the familiar saw-tooth of hard resets, whereas the HyperLIF membrane rises smoothly, hovers near threshold, and is discharged continuously by the gate. A revealing detail is that the gate completes several readiness cycles while only a single upward threshold crossing is registered: the continuous reset keeps the trajectory near, but mostly below, the event level, so the event channel underreports the activity that the gate state makes explicit. This already motivates treating $r(t)$ as an observable in its own right (**??**).

B. Fixed vs. Adaptive Reset Conductance

?? drives both continuous-reset models with a stress current that ramps to $J = 4$. During the high-current phase the fixed-gain HyperLIF membrane latches above threshold (its gate saturates and its conductance plateaus at $G_b = 100$), i.e. it enters the DB. Adaptive HyperLIF instead raises its conductance with $|J|$ (the dynamic gain G_{dyn} tracks the staircase input) and keeps oscillating below threshold. This is the time-domain signature of the operating-range expansion characterized statically next.

C. Frequency–Current Response and the Depolarization Block

?? and **??** summarize the steady-state response to constant current. Three regimes are evident. LIF firing increases monotonically and without bound (runaway). Fixed-gain HyperLIF fires only over a narrow band ($J \in [1.5, 2.2]$) and then collapses to zero at $J \approx 2.3$–2.4, close to the open-gate branch estimate $J_{\text{DB}} = 2.4$ from **??**; beyond it the mean membrane voltage climbs linearly as $v^* = J/(1+\tau G_b) = J/3$ (e.g. 1.000 at $J = 3$, 1.333 at $J = 4$), and the gate saturates—the high-voltage latch of **??**. The fixed-point prediction matches the simulated latched voltage to all reported

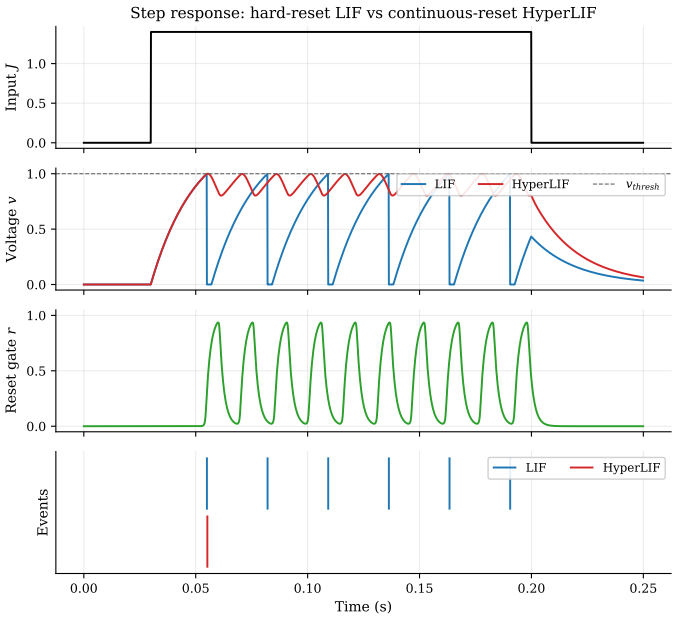


Fig. 2. Step response. Hard-reset LIF (blue) vs. continuous-reset HyperLIF (red): input current, membrane voltage with threshold, the HyperLIF reset gate $r(t)$, and event rasters. The gate cycles several times while only one HyperLIF event is registered. Source: `01_lif_vs_hyperlif.csv`.

TABLE III
STEADY-STATE RESPONSE AT REPRESENTATIVE CONSTANT CURRENTS (FROM `03_current_sweep.csv`). THE FIXED-GAIN HYPERLIF FIRING RATE COLLAPSES TO ZERO AND ITS MEAN VOLTAGE LATCHES ABOVE THRESHOLD ONCE J EXCEEDS THE ANALYTICAL DEPOLARIZATION-BLOCK ONSET $J_{\mathrm{DB}} = 2.4$, WHEREAS ADAPTIVE HYPERLIF SUSTAINS SPIKING WITH A BOUNDED MEAN VOLTAGE.

	Firing rate (Hz)			Mean voltage	
J	LIF	HyperLIF	Adaptive	HyperLIF	Adaptive
1.0	0.0	0.0	0.0	0.906	0.906
1.5	42.2	71.1	0.0	0.908	0.903
2.0	62.2	71.1	102.2	0.917	0.910
2.4	80.0	0.0	117.8	0.840	0.912
3.0	100.0	0.0	135.6	1.000	0.913
4.0	128.9	0.0	0.0	1.333	0.910

digits (1.3333 vs. 1.3333 at $J = 4$). Adaptive HyperLIF fires over a substantially wider band ($J \in [1.6, 3.5]$) with a higher peak rate (151 vs. 73 Hz) and a *bounded* mean voltage near 0.9, never exhibiting the high-voltage latch, consistent with **????** ($v^*_\infty = 0.625 < 0.8$). The adaptive model is not unconditionally firing: with finite β it has its own, higher operating ceiling ($J \gtrsim 3.6$ here) above which the bounded oscillation no longer crosses the default event surface and detected spiking ceases. Crucially, this failure is bounded—the membrane oscillates below threshold near 0.9 rather than latching at $J/3$—so the event channel becomes silent while the continuous state remains dynamically active.

?? isolates the contrast at a single sustained drive $J = 3.3$ (above J_{DB}): the fixed-gain neuron settles to the predicted latched voltage 1.1 with a saturated gate and emits no further events, while the adaptive neuron produces a clean limit cycle with 42 events over 0.3 s.

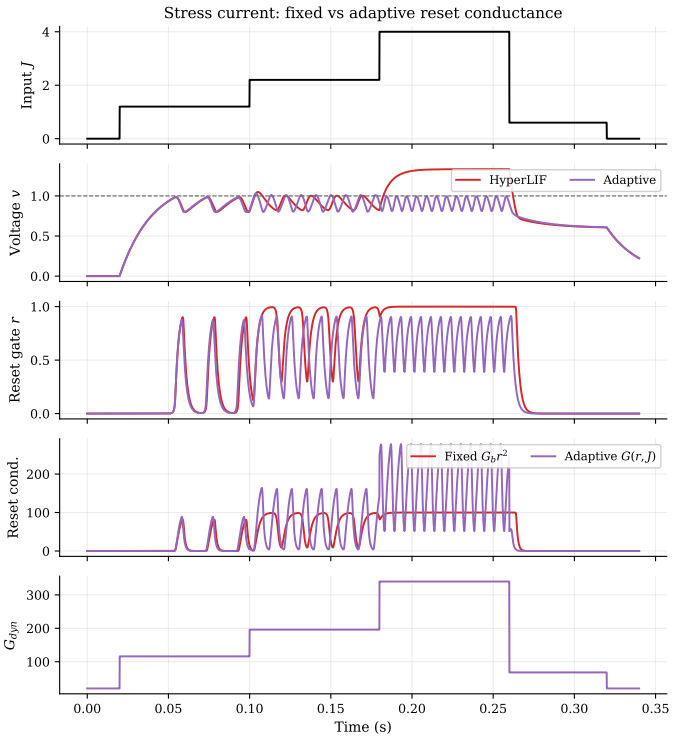


Fig. 3. Stress current. Fixed-gain HyperLIF (red) latches during the high-current phase while Adaptive HyperLIF (purple) sustains oscillation; the dynamic gain $G_{\mathrm{dyn}} = G_{\mathrm{base}} + \beta|J|$ follows the input staircase. Source: `02_hyperlif_vs_adaptive.csv`.

TABLE IV
VOLTAGE ERROR OF THE TWO STEPPING SCHEMES AGAINST A FINE RK4 REFERENCE OF THE CONTINUOUS ODE (FROM `06_integration_accuracy.csv`). BOTH CONVERGE AT FIRST ORDER; THE GATE UPDATE IS SHARED AND IDENTICAL.

Δt (s)	RMSE explicit	RMSE exponential	Gate RMSE
2.000×10^{-3}	0.0237	0.0877	0.16
1.000×10^{-3}	0.0307	0.0609	0.2
5.000×10^{-4}	0.0204	0.0366	0.134
2.500×10^{-4}	0.0118	0.0202	0.0806
1.000×10^{-4}	0.00522	0.0086	0.0367
5.000×10^{-5}	0.0027	0.00439	0.0191

D. Numerical Accuracy and Stability

?? and **??** report the membrane error of both stepping schemes against the RK4 reference. Both converge at first order in Δt, and the gate error is identical because the gate update **??** is shared. In the convergent regime the explicit and exponential schemes have comparable accuracy (here the explicit scheme has a slightly lower trajectory RMSE), so the value of the exponential scheme is not accuracy but *stability*: **??**(b) drives the gate fully open ($J = 4$, $\lambda \approx 150$) and increases Δt; once $\lambda \Delta t > 2$ the explicit scheme develops growing spurious overshoot (its maximum $|v|$ rises to $2, 3, 4, 6$ as $\lambda \Delta t$ grows), whereas the exponential scheme remains bounded near the true latched value $v^* = J/3$. This is why the Nengo port, which must remain well-behaved at arbitrary Δt and high conductance, uses the exponential update.

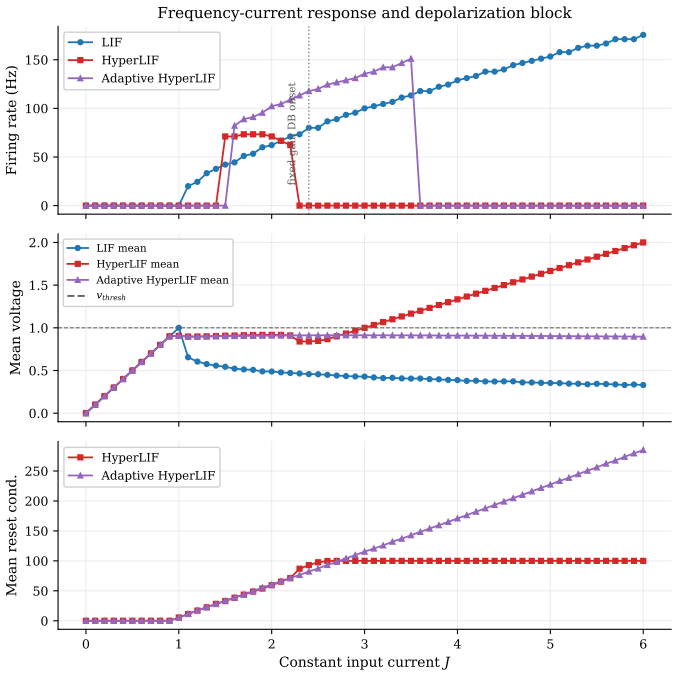


Fig. 4. Frequency–current response. (Top) firing rate: LIF runs away, fixed-gain HyperLIF collapses near the open-gate branch estimate $J_{\mathrm{DB}} = 2.4$, Adaptive HyperLIF sustains a wider band. (Middle) mean voltage: the HyperLIF latch climbs as $J/3$ while Adaptive stays bounded near 0.9. (Bottom) mean reset conductance saturates for HyperLIF but grows with J for Adaptive. Source: `03_current_sweep.csv`.

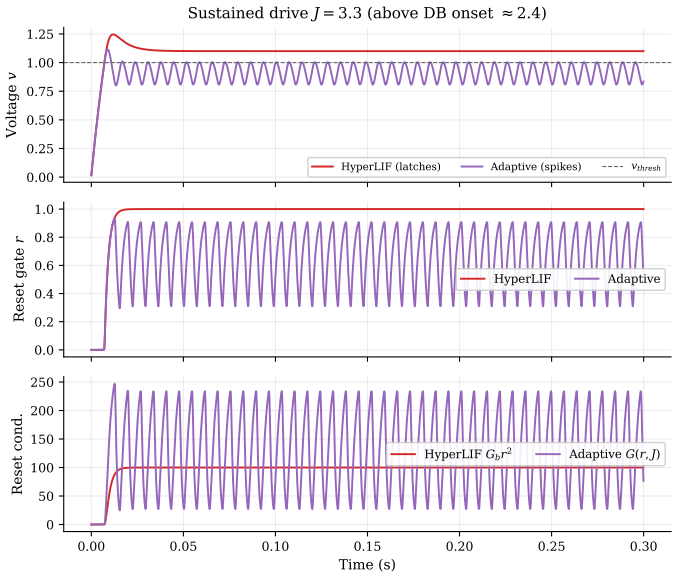


Fig. 5. Depolarization-block latch at sustained drive $J = 3.3$. Fixed-gain HyperLIF (red) latches at $v \approx 1.1$ with a saturated gate; Adaptive HyperLIF (purple) maintains a limit cycle. Source: `04_depolarization_block.csv`.

E. Python–Nengo Parity

?? and **??** report the central validation result. The standalone exponential stepping and the Nengo custom-neuron stepping produce identical voltage and gate traces, and identical event timing, for both models and both protocols: the voltage and gate RMSE are exactly 0, the maximum absolute voltage/gate difference is exactly 0, and the event-mismatch count is 0. The

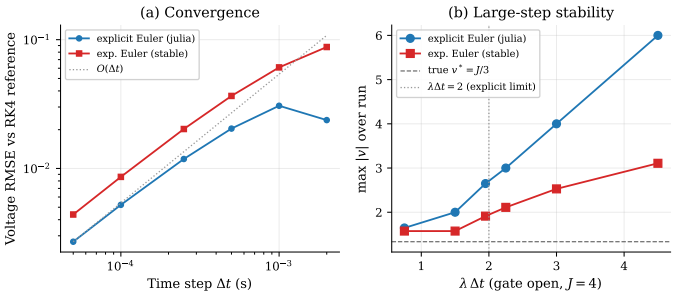


Fig. 6. (a) First-order convergence of both schemes against an RK4 reference of the continuous ODE. (b) Large-step stability at $J = 4$: explicit Euler overshoots once $\lambda \Delta t > 2$; the exponential update stays bounded near $v^* = J/3$. Source: `06_integration_accuracy.csv`, `06_stability.csv`.

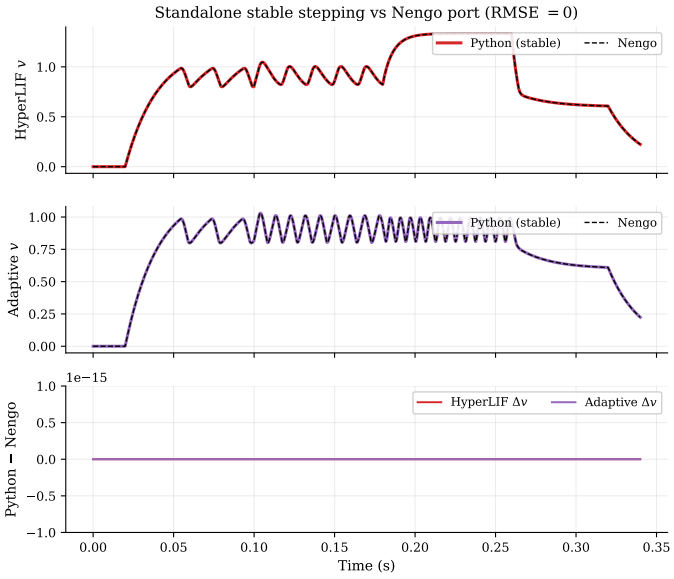


Fig. 7. Standalone stable stepping (thick colored) vs. the Nengo port (dashed black) for HyperLIF and Adaptive HyperLIF voltage, with the difference trace (bottom) flat at machine precision. Source: `07_python_vs_nengo.csv`, `07_parity_metrics.csv`.

difference trace in **??** is flat at the 10^{-15} scale. Because both implementations call the same numerical kernel (**??**), this is an algorithmic identity, not a statistical fit—the strongest form of evidence that the Nengo port *is* the implemented model rather than an approximation of it.

F. Gate-State Decoding

?? and **??** compare scalar linear decoders fit to three internal signals of an Adaptive HyperLIF neuron driven by a smooth current. A voltage decoder recovers the input current with test RMSE 0.258; a decoder of exponentially filtered events does worse (0.321), reflecting the sparsity of the event channel; and a decoder of the reset gate recovers a readiness target with test RMSE 0.287. The gate decoder is comparable to the voltage decoder even though the raw gate is a fast, spiky signal, indicating that the gate carries genuine, decodable information about proximity to the reset transition.

TABLE V
STANDALONE STABLE STEPPING VS. THE NENGO NEURONTYPE PORT (FROM 07_PARITY_METRICS.CSV). AGREEMENT IS EXACT TO FLOATING POINT: ZERO RMSE, ZERO MAXIMUM VOLTAGE/GATE DIFFERENCE, AND ZERO EVENT-TIMING MISMATCHES ACROSS BOTH PROTOCOLS AND BOTH MODELS.

Model	Protocol	v RMSE	$\max \lvert\Delta v\rvert$	r RMSE	Event mism.
HyperLIF	stress	0	0	0	0
Adaptive	stress	0	0	0	0
HyperLIF	step	0	0	0	0
Adaptive	step	0	0	0	0

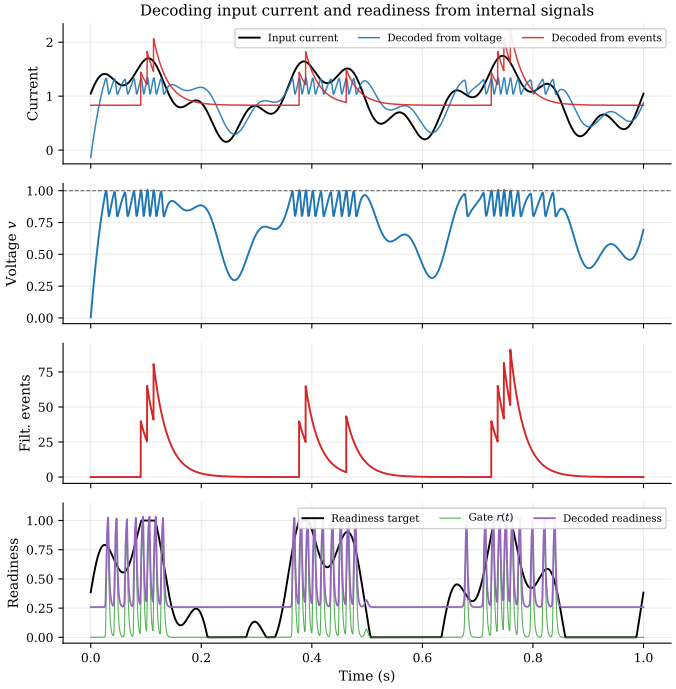


Fig. 8. Decoding from internal signals. (Top) input current and its linear decodes from voltage and from filtered events. (Middle panels) the voltage and filtered-event features. (Bottom) the readiness target, the raw gate $r(t)$, and the gate-based readiness decode. Source: 05_gate_state_decoding.csv, 05_decoding_metrics.csv.

VIII. INTERPRETATION: GATE-STATE READINESS-TO-FIRE

The reset gate is more than an internal bookkeeping variable. By **????**, r tracks $\sigma(\alpha(v-(v_{\text{thresh}}-\gamma r)))$: it rises as the membrane approaches the (hysteresis-lowered) threshold and saturates once the reset trajectory is committed. It is therefore a continuous *readiness-to-fire* signal: small r means far from the reset transition, rising r means approaching it, and saturated r means the reset is fully engaged. Two observations from **??** make this concrete. First, in **??** the gate completes several readiness cycles while the event channel reports a single spike—the gate exposes sub-threshold structure that spikes discard. Second, **??** shows that a linear gate decoder is competitive with a voltage decoder for a readiness target. Unlike the voltage, whose interpretation near reset is ambiguous in a hard-reset neuron, the gate is a bounded $[0,1]$ quantity with a direct interpretation in terms of the effective threshold. This makes $r(t)$ a natural *residual feature* to augment spike/rate features in downstream estimation.

TABLE VI
SCALAR LINEAR DECODERS FIT BY LEAST SQUARES WITH A 70/30 TRAIN/TEST SPLIT (FROM 05_DECODING_METRICS.CSV).

Decoder	Train RMSE	Test RMSE
Voltage $\rightarrow$ input current	0.293	0.258
Filtered events $\rightarrow$ input current	0.347	0.321
Reset gate $\rightarrow$ readiness	0.283	0.287

IX. CONNECTION TO RESIDUAL AUGMENTATION AND QUATERNION ATTITUDE CONTROL

We now outline the intended application, and we are explicit that this section is *forward-looking*: no closed-loop control results are claimed here. The NEF pipeline that Nengo implements suggests a direct route to using HyperLIF neurons inside an attitude controller. An attitude error—for example a quaternion error vector between the estimated and desired orientations [**?**]—is encoded as input current J to a HyperLIF ensemble; the ensemble's spikes, membrane voltages, and reset gates are decoded into control or residual features; and a controller maps those features to torques. NEF-based spiking networks have already been shown to learn adaptive control signals online [**?**], and the gate provides an additional, interpretable channel:

$$\text{error} \rightarrow J \rightarrow (v, r, \text{spikes}) \rightarrow \text{decoded residual} \rightarrow \text{torque}. \tag{17}$$

The hypothesis to be tested in future work is that a gate-state residual term improves tracking or robustness relative to spike-only or voltage-only decoding, because $r(t)$ encodes readiness information that the event channel omits (**??**). The validated NeuronType port and the exact parity result of **??** are precisely what make such a study trustworthy: any controller behavior can be attributed to the model, not to an implementation discrepancy. The present paper deliberately keeps the neuron reference free of controller code; the controller stage will import this reference rather than re-define the equations.

X. LIMITATIONS

We list the limitations plainly. (i) The validation is performed on a single neuron driven by injected current; ensemble-level NEF decoding and closed-loop control are not evaluated here. (ii) The parameters in **??** are a normalized reference set chosen for clean demonstration; they differ from the benchmark-scale training parameters of [**?**], so the specific numbers ($J_{\text{DB}} = 2.4$, $\beta_{\text{crit}} = 62.5$, etc.) are parameter-dependent instances of the general relations **????**, not universal constants. (iii) The DB analysis uses a frozen-gate fixed-point argument; it correctly predicts the onset and latched voltage observed in simulation but is a sufficient-condition characterization rather than a full bifurcation analysis, and the adaptive ceiling above $J \approx 3.6$ is reported empirically. (iv) The decoders are deliberately simple scalar least-squares fits on a single neuron; the raw gate is spiky and its decode

is correspondingly noisy. (v) The attitude-control connection of **??** is a design and a hypothesis, not a result. (vi) The explicit-Euler scheme, included for reference fidelity, is only conditionally stable (**??**).

XI. Future Work

The natural next steps are: (a) NEF ensemble experiments—tuning-curve calibration, multi-neuron encoding/decoding, and noise robustness for the gate channel; (b) closed-loop quaternion attitude control following **??**, with ablations isolating spike-only, voltage-only, gate-only, and combined decoders; (c) repeated, seed-matched trials with paper-scale statistics; (d) mapping the gate $r(t)$ to local, multi-factor plasticity rules as a low-memory alternative to backpropagation through time, as suggested in [**?**]; and (e) deployment considerations on event-driven neuromorphic hardware [**?**], where the intrinsic activity bounding of the adaptive model is attractive.

XII. Conclusion

We presented a clean, equation-faithful, and rigorously validated reference implementation of the HyperLIF and Adaptive HyperLIF continuous-reset neurons, compatible with Nengo. Every manuscript equation is realized in code; a frozen-open-gate analysis gives the branch estimate $J_{\mathrm{DB}} = (v_{\mathrm{thresh}} - \gamma)(1+\tau G_b)$ and branch voltage $J/(1+\tau G_b)$; the latter matches the simulated high-drive latch. The adaptive conductance law bounds the positive-drive, $r = 1$ asymptotic branch below the gate knee when $\beta > 1/(\tau(v_{\mathrm{thresh}} - \gamma))$, changing the high-drive regime without proving global stability or indefinite firing. The two stepping schemes are characterized for accuracy and stability; and the Nengo port reproduces the standalone model to floating-point exactness. Interpreting the reset gate as a decodable readiness-to-fire signal, we outlined a concrete, validated-substrate path toward gate-state residual decoding for quaternion attitude control, which we leave as future work. The complete project regenerates every figure, table, and number from the included code and data.

Appendix A
Parameters

?? lists the default parameters, reflected directly from the `LIFParams`, `HyperLIFParams`, and `AdaptiveHyperLIFParams` dataclasses, so the table cannot drift from the implementation.

TABLE VII
DEFAULT PARAMETERS OF THE REFERENCE IMPLEMENTATION, REFLECTED DIRECTLY FROM THE LIFPARAMS, HYPERLIFPARAMS, AND ADAPTIVEHYPERLIFPARAMS DATACLASSES. ALL VOLTAGES AND GAINS ARE IN THE NORMALIZED CONVENTION ($v_{\mathrm{rest}} = v_{\mathrm{reset}} = 0$, $v_{\mathrm{thresh}} = 1$); J AND G ARE DIMENSIONLESS RELATIVE TO THE LEAK.

Parameter	Symbol	LIF	HyperLIF	Adaptive
Membrane time constant	τ (s)	0.02	0.02	0.02
Refractory period	τ_{ref} (s)	0.002	–	–
Resting potential	v_{rest}	0	0	0
Reset potential	v_{reset}	0	0	0
Threshold	v_{thresh}	1	1	1
Gate steepness	α	–	100	100
Gate time constant	τ_r (s)	–	0.002	0.002
Hysteresis factor	γ	–	0.2	0.2
Fixed reset gain	G_b	–	100	–
Baseline reset gain	G_{base}	–	–	20
Adaptive scaling	β	–	–	80
Spike amplitude	A	1	1	1
Min. voltage clamp	v_{min}	0	0	0

Appendix B
Equation-to-Code Map

???? correspond to `src/neurons/lif.py` (`_lif_voltage_update`, `simulate_lif`). **??????????????** correspond to `src/neurons/hyperlif.py` (`gate_target`, `exact_gate_update`, `stable_voltage_update`, `HyperLIFNeuron.step`). **????** correspond to `src/neurons/adaptive_hyperlif.py`. The Nengo realization of **??** is in `src/neurons/nengo_hyperlif.py` (`HyperLIFNengo.step`, `AdaptiveHyperLIFNengo.step`). **????????** are implemented and validated in `src/analysis/depolarization_block.py` and exercised by `experiments/03_current_sweep.py`. A line-level table is maintained in the accompanying `MATH_CODE_MAP.md`.

Appendix C
Reproducibility

From the project root, after creating the environment (`scripts/setup_env.ps1`): run `python experiments/run_all.py`, `python figures/make_figures.py`, `python tables/make_tables.py`, and `scripts/build_paper.ps1`. This regenerates all CSV/logs, all figures (PNG + PDF), all LaTeX tables, and compiles this document. See `REPRODUCIBILITY.md` and `RUNNERS.md` for details and `RESULTS_AUDIT.md` for the validated/exploratory and paper-ready/future-work classification of every claim.